If you skipped Chapter 5, *Render Native Screens with SwiftUI*, then make sure you add the following to your Rails app at `app/views/maps/show.json.jbuilder`. This provides the JSON needed for each hike's map.

ch05_10/rails/app/views/maps/show.json.jbuilder

```
json.extract! @hike, :name, :latitude, :longitude
```

Create a new package named `viewmodels` and add a Kotlin file named `HikeViewModel.kt` that implements `ViewModel`. Initialize this with a URL `String`—this will point to the JSON endpoint that exposes the hike's attributes.

ch06_22/android/app/src/main/java/com/masilotti/hikingjournal/viewmodels/HikeViewModel.kt

```
package com.masilotti.hikingjournal.viewmodels

import androidx.lifecycle.ViewModel

class HikeViewModel(private val url: String) : ViewModel() {
}
```

Next, add a property to the view model that references an instance of `Hike`. Using `mutableStateOf()` ensures that when this property is updated, the changes will propagate to the view, just like the `@Observable` keyword we used in Swift.

ch06_23/android/app/src/main/java/com/masilotti/hikingjournal/viewmodels/HikeViewModel.kt

```
  package com.masilotti.hikingjournal.viewmodels

» import androidx.compose.runtime.mutableStateOf
  import androidx.lifecycle.ViewModel
» import com.masilotti.hikingjournal.models.Hike

  class HikeViewModel(private val url: String) : ViewModel() {
»     var hike = mutableStateOf<Hike?>(null)
»         private set
```